UNIVERSIDAD PONTIFICIA CATÓLICA DEL ECUADOR

SEDE MANABÍ

Base de Datos en la Nube

Práctica de configuración de seguridad en una base de datos

Santiago Esquetini

Ing. Alex Fernando Ricaurte Segovia

Martes, 13 de octubre de 2026

1. Objetivo

Diseñar e implementar una arquitectura de red que garantice comunicación privada entre una máquina virtual de aplicación y una máquina virtual de base de datos, utilizando buenas prácticas de seguridad y segmentación de red, de modo que la aplicación se conecte a la base de datos de forma segura y que esta última no sea accesible desde Internet.

La guía plantea la solución en Microsoft Azure (VNet, subredes y Network Security Group). En esta práctica se implementó la misma arquitectura en local con **Vagrant** y **VMware Workstation**, usando tres máquinas virtuales: **router**, **app** y **db**.

2. Desarrollo

2.1. Arquitectura propuesta

Cada elemento de Azure se reemplazó por su equivalente en el laboratorio local:

Guía de Azure	Implementación local
Virtual Network 10.0.0.0/16	Redes privadas de VMware `vmnet2` y `vmnet3`
Subred de la aplicación 10.0.1.0/24	`vmnet2` → VM app con IP `10.0.1.10`
Subred de la base de datos 10.0.2.0/24	`vmnet3` → VM db con IP `10.0.2.10`
Enrutamiento interno de la VNet	VM router (`10.0.1.1` / `10.0.2.1`) con `ip_forward = 1`
VM de la BD sin IP pública	MySQL escucha solo en la IP privada `10.0.2.10`
Network Security Group de la BD	Firewall UFW: solo `10.0.1.0/24` → puerto `3306/tcp`
Conexión por IP privada	`mysql -h 10.0.2.10` desde la VM app

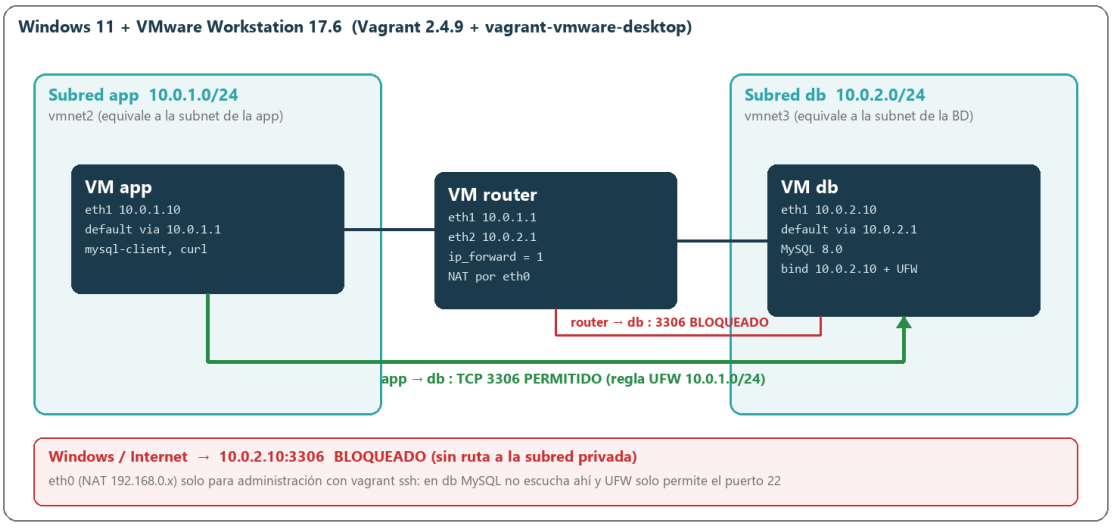


Figura 1. *Arquitectura implementada: dos subredes separadas, un router entre ellas y el puerto 3306 de la base de datos abierto solo para la subred de la aplicación.*

2.2. Instalación de las herramientas

Se utilizó VMware Workstation 17.6, que ya estaba instalado en el equipo. Se instaló **Vagrant 2.4.9** con `winget install Hashicorp.Vagrant`, el servicio **Vagrant VMware Utility 1.0.24** y el plugin del proveedor con `vagrant plugin install vagrant-vmware-desktop`.

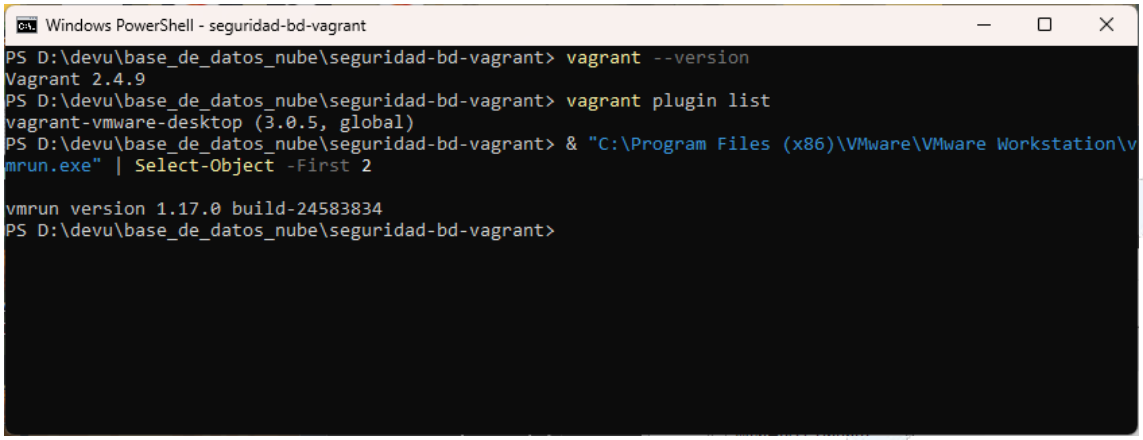


Figura 2. *Versiones de Vagrant, del plugin vagrant-vmware-desktop y de vmrun (VMware Workstation).*

> **CORRECTO:** *Vagrant quedó conectado a VMware Workstation mediante el plugin* `vagrant-vmware-desktop`.

2.3. Creación de la red virtual y sus subredes

En lugar de la VNet de Azure se crearon dos redes privadas en VMware con la herramienta `vnetlib64.exe`, como Administrador: **vmnet2** para la subred de la aplicación (10.0.1.0/24) y **vmnet3** para la subred de la base de datos (10.0.2.0/24). No tienen DHCP ni IP del equipo anfitrión dentro de ellas, así que solo las máquinas virtuales conectadas se comunican entre sí.

host/crear_redes_vmware.ps1 (fragmento)

```
vnetlib64.exe -- add adapter vmnet2
vnetlib64.exe -- set vnet vmnet2 mask 255.255.255.0
vnetlib64.exe -- set vnet vmnet2 addr 10.0.1.0     # subred de la app
vnetlib64.exe -- set vnet vmnet3 mask 255.255.255.0
vnetlib64.exe -- set vnet vmnet3 addr 10.0.2.0     # subred de la base de datos
```

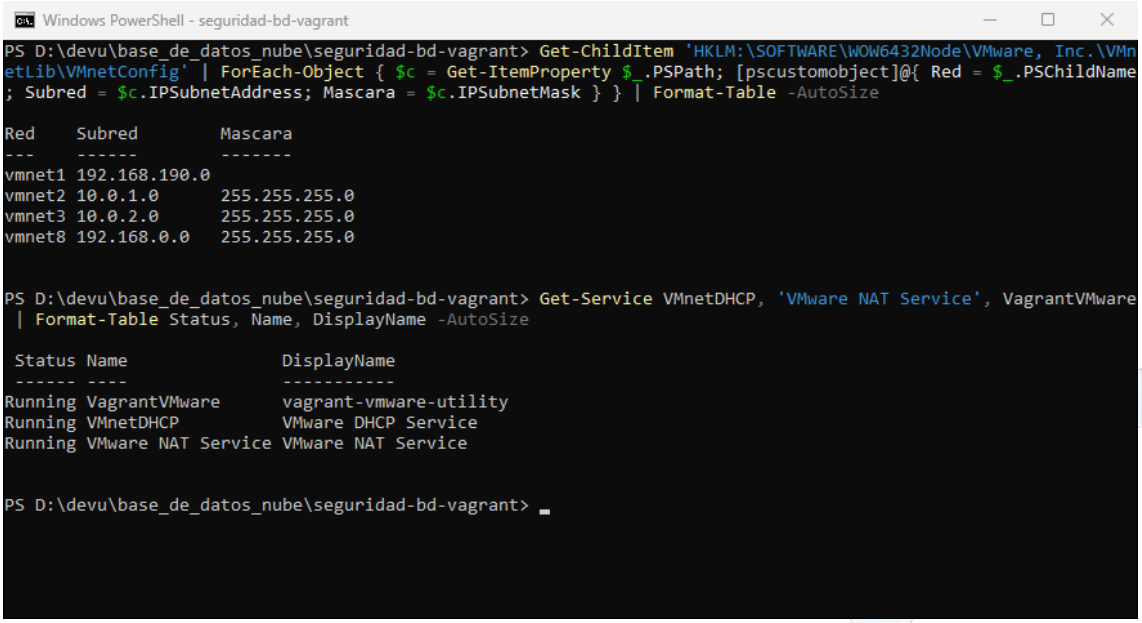


Figura 3. *Redes de VMware creadas (vmnet2 = 10.0.1.0, vmnet3 = 10.0.2.0) y servicios DHCP, NAT y Vagrant VMware Utility en ejecución.*

> **CORRECTO:** *Las subredes 10.0.1.0/24 y 10.0.2.0/24 quedaron separadas en redes virtuales distintas.*

2.4. Creación de las máquinas virtuales con Vagrant

Las tres máquinas se definieron en un único `Vagrantfile` con la box `bento/ubuntu-22.04`. Al asignar una IP de la subred 10.0.1.x, el plugin conecta esa tarjeta a vmnet2; con una IP 10.0.2.x, la conecta a vmnet3. Cada VM tiene un script de aprovisionamiento en la carpeta `provision/`.

Vagrantfile (fragmento)

```
config.vm.box = "bento/ubuntu-22.04"

config.vm.define "router" do |r|
  r.vm.network "private_network", ip: "10.0.1.1"   # subred app
  r.vm.network "private_network", ip: "10.0.2.1"   # subred db
  r.vm.provision "shell", path: "provision/router.sh"
  r.vm.provision "shell", path: "provision/router_nat.sh", run: "always"
end

config.vm.define "app" do |a|
  a.vm.network "private_network", ip: "10.0.1.10"
  a.vm.provision "shell", path: "provision/ruta_default.sh", args: ["10.0.1.1"], run:
"always"
  a.vm.provision "shell", path: "provision/app.sh"
end

config.vm.define "db" do |d|
  d.vm.network "private_network", ip: "10.0.2.10"
  d.vm.provision "shell", path: "provision/ruta_default.sh", args: ["10.0.2.1"], run:
"always"
  d.vm.provision "shell", path: "provision/db.sh"
end
```

Como el Vagrantfile de la guía estaba hecho para VirtualBox, se adaptó a VMware:

Original (VirtualBox)	Adaptación (VMware)	Motivo
`ubuntu/jammy64`	`bento/ubuntu-22.04`	La box de Ubuntu solo existe para VirtualBox
`virtualbox__intnet`	`vmnet2` / `vmnet3`	VMware usa vmnets en vez de redes internas
Router con 512 MB	Router con 1024 MB	Con 512 MB el kernel daba *kernel panic* al arrancar
MASQUERADE por `enp0s3`	Interfaz detectada (`eth0`)	En VMware la interfaz NAT se llama `eth0`
UFW sin regla para SSH	SSH permitido solo en `eth0`	Sin la regla, `vagrant ssh db` queda bloqueado

Se levantaron las máquinas con `vagrant up`. En el aprovisionamiento de la VM db se instala MySQL, se aplican las reglas del firewall y se crea la base de datos:

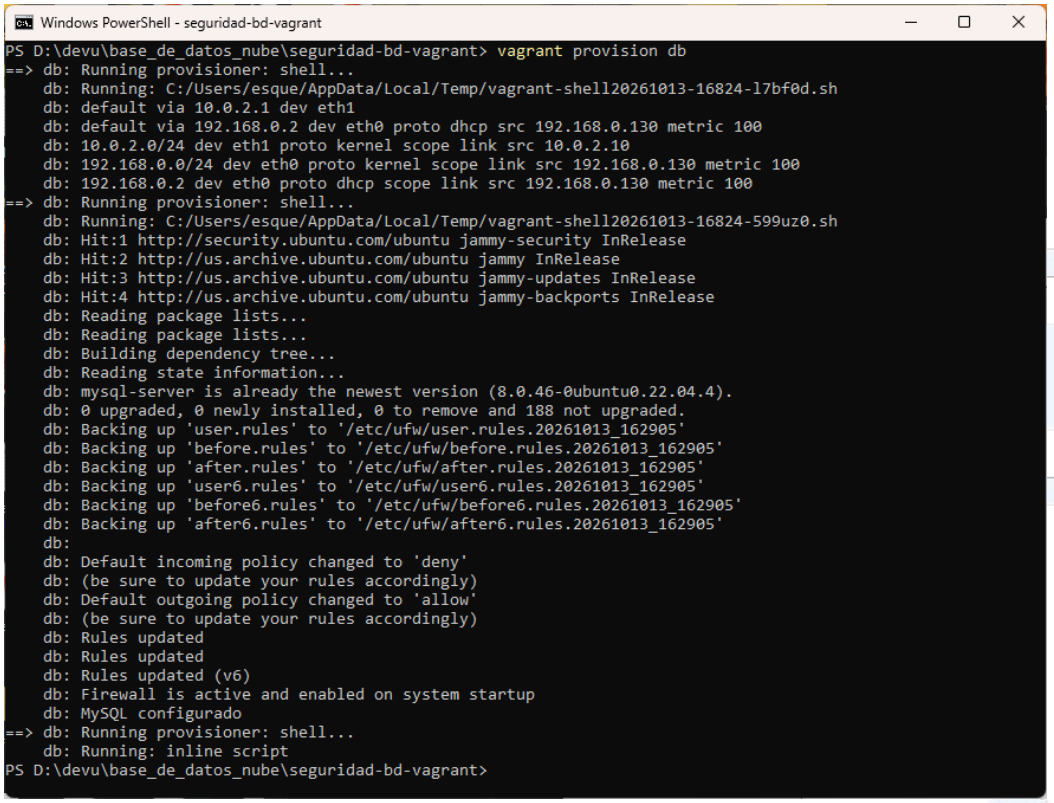


Figura 4. *Aprovisionamiento de la VM db: ruta por defecto vía 10.0.2.1, MySQL instalado, reglas de UFW aplicadas y «MySQL configurado».*

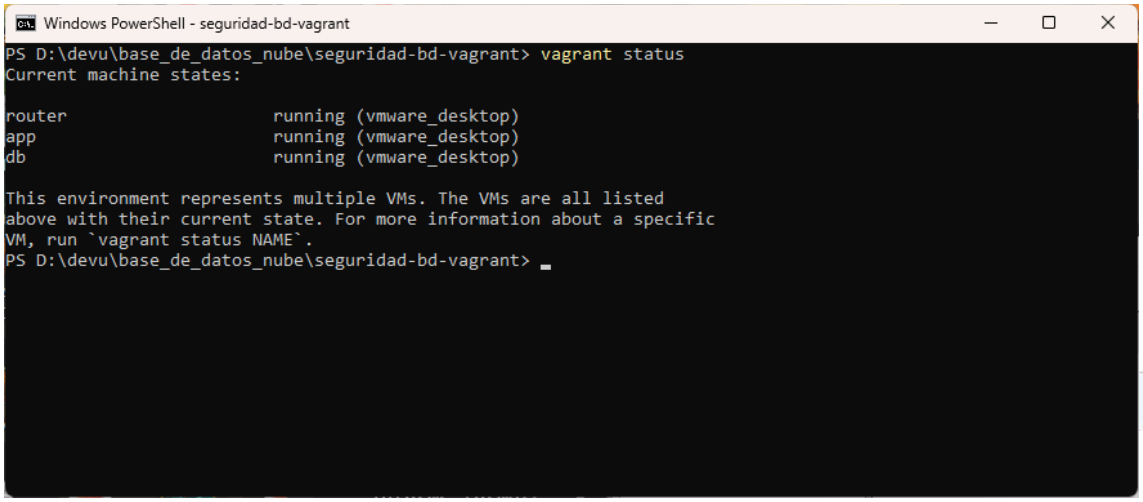


Figura 5. *vagrant status: las tres máquinas en estado running (vmware_desktop).*

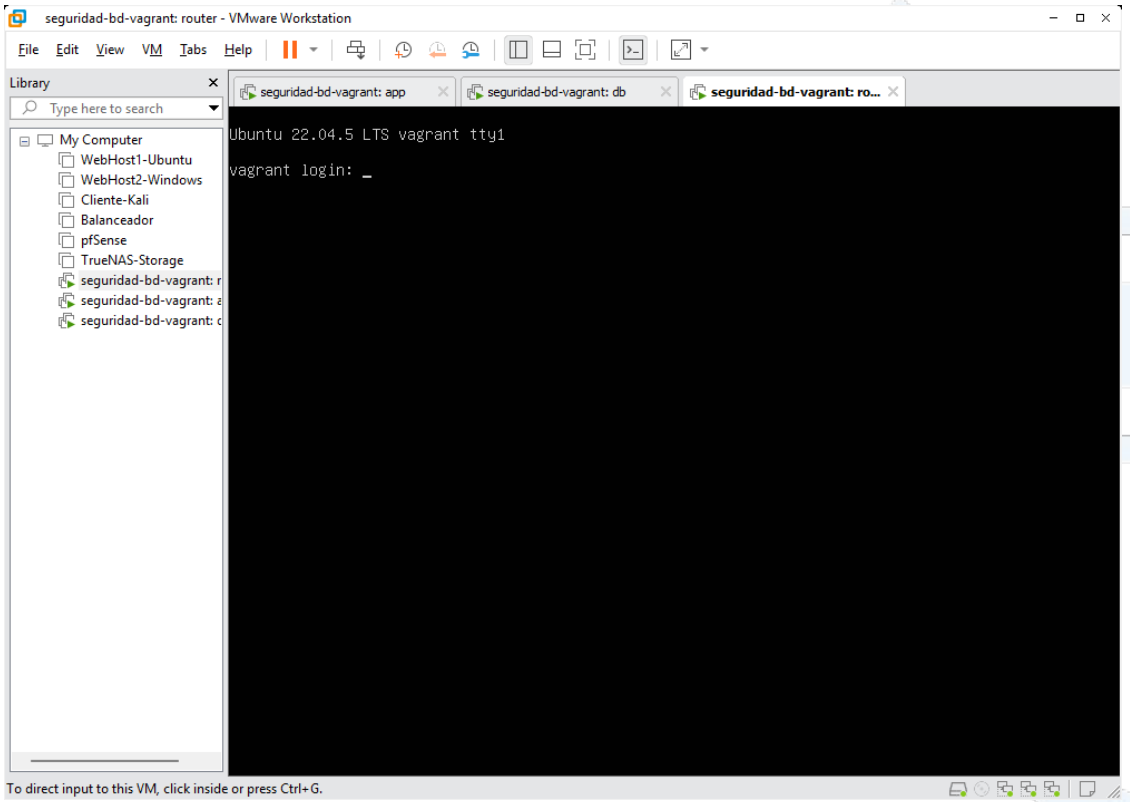


Figura 6. *Las tres VMs (app, db y router) encendidas en VMware Workstation.*

CORRECTO: *Las tres máquinas virtuales quedaron creadas, aprovisionadas y en ejecución.*

2.5. Configuración del router

El router tiene una interfaz en cada subred y el reenvío de paquetes activado, que es lo que en Azure hace la VNet de forma automática. También aplica NAT (MASQUERADE) para que app y db puedan salir a Internet a descargar paquetes.

Verificación

```
vagrant ssh router
ip a                            # debe mostrar 10.0.1.1 y 10.0.2.1
sysctl net.ipv4.ip_forward      # debe devolver = 1
```

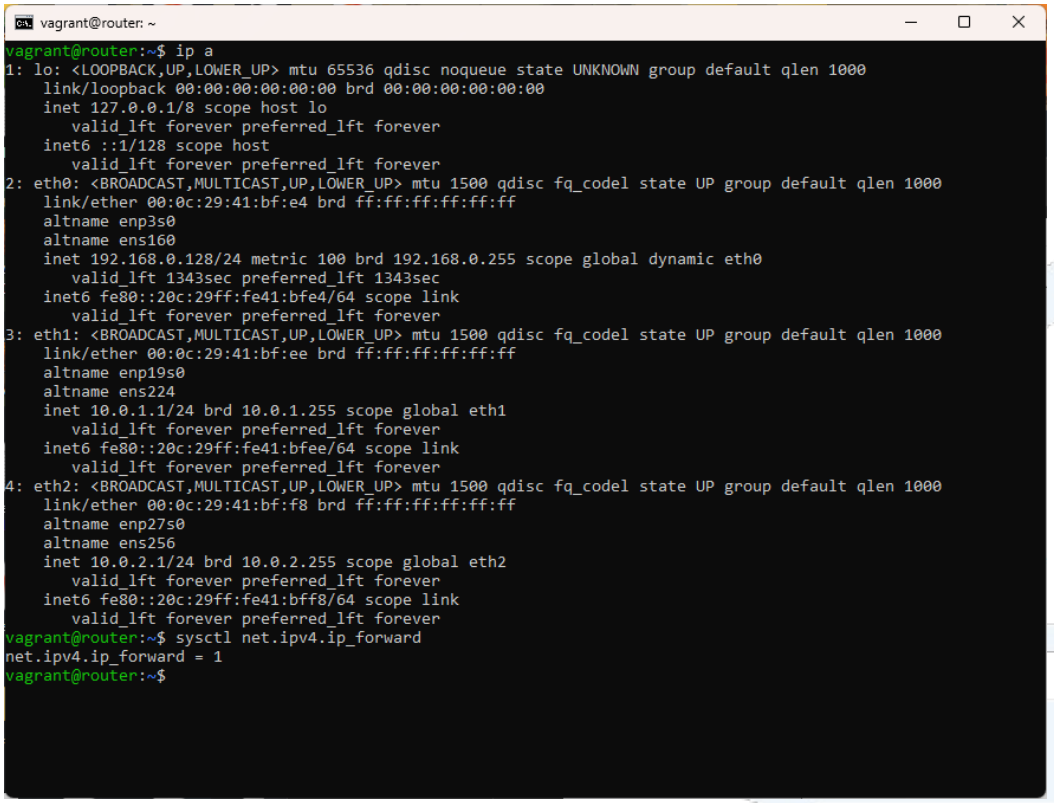


Figura 7. *El router tiene eth1 = 10.0.1.1/24 y eth2 = 10.0.2.1/24, y net.ipv4.ip_forward = 1.*

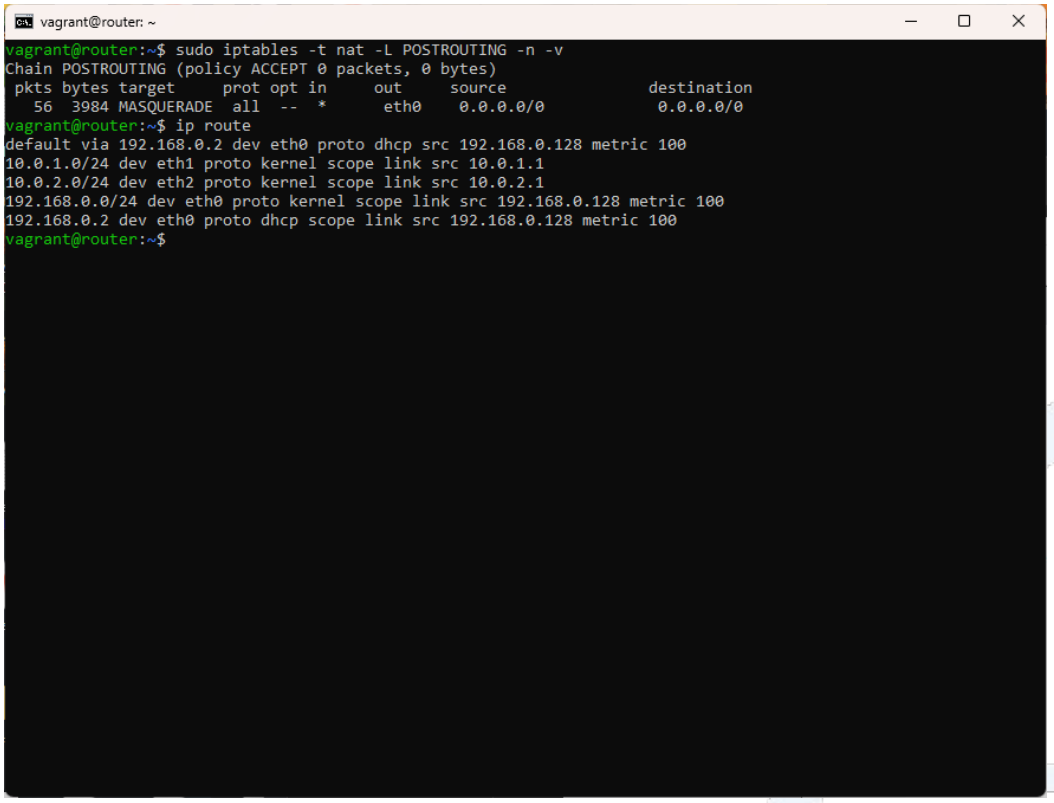


Figura 8. *Regla MASQUERADE por eth0 y tabla de rutas del router con las dos subredes conectadas.*

> **CORRECTO:** *El router tiene las IP 10.0.1.1 y 10.0.2.1, y el reenvío de paquetes está activo.*

2.6. Configuración de la máquina de la aplicación

La VM app tiene la IP privada 10.0.1.10, y su ruta por defecto apunta al router (10.0.1.1). Así llega a la subred de la base de datos y a Internet.

Verificación

```
vagrant ssh app
ip a                    # debe mostrar 10.0.1.10
ip route                # default via 10.0.1.1
ping -c 2 10.0.2.10     # debe responder
```

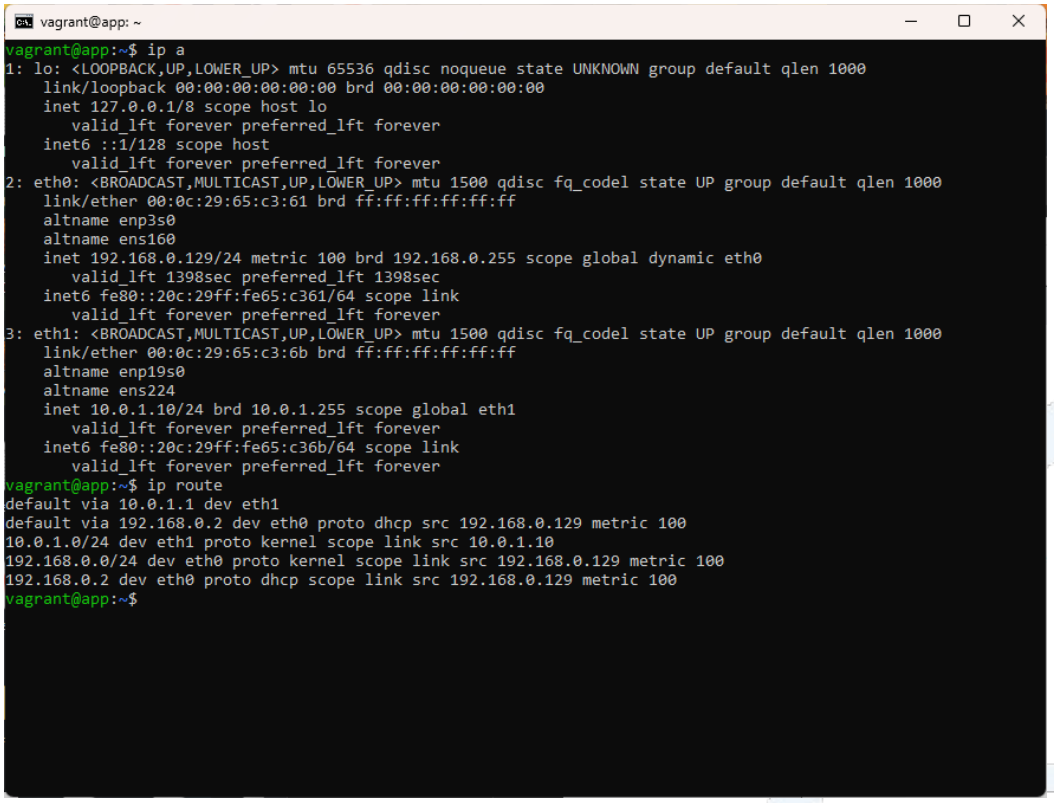


Figura 9. *La VM app con IP 10.0.1.10/24 y ruta por defecto default via 10.0.1.1 dev eth1.*

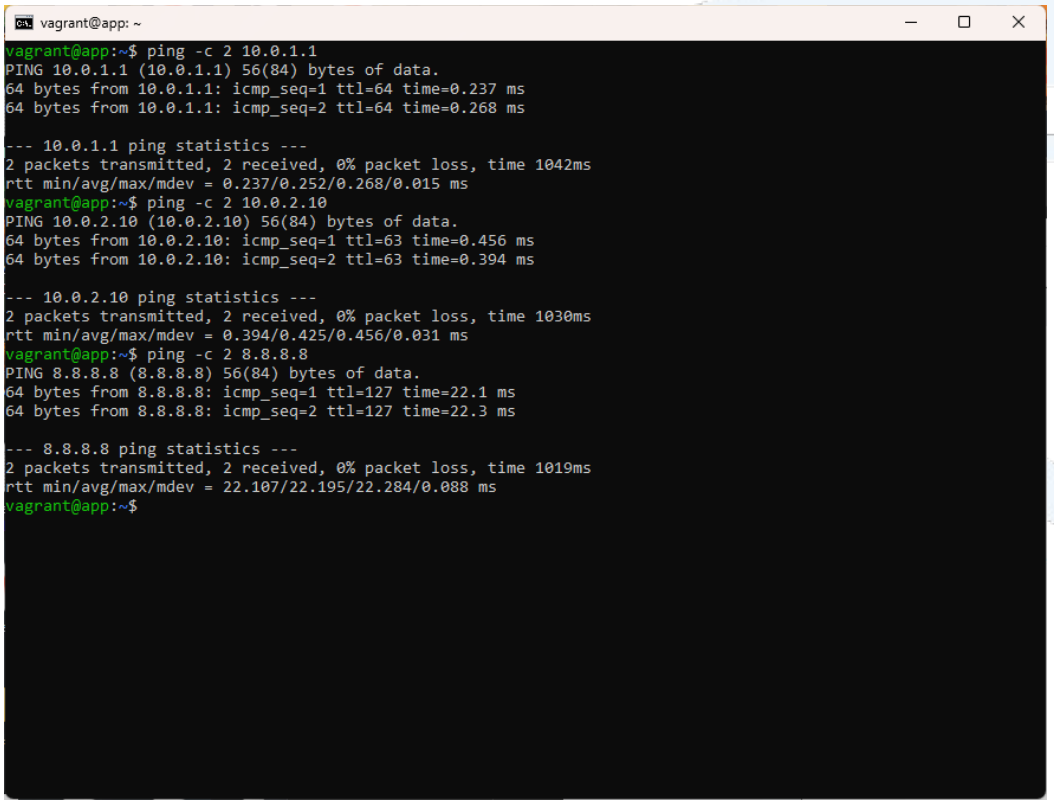


Figura 10. *Ping desde app al router (10.0.1.1), a la base de datos en otra subred (10.0.2.10, ttl=63 porque pasa por el router) y a Internet (8.8.8.8).*

CORRECTO: *La aplicación llega a la base de datos por su IP privada, pasando por el router.*

2.7. Configuración de la base de datos y del Network Security Group

En la VM db se configuró MySQL para escuchar **solo en la IP privada 10.0.2.10** (`bind-address`). El firewall UFW hace el papel del NSG de Azure: deniega toda conexión entrante y solo permite el puerto 3306 desde la subred de la aplicación.

provision/db.sh (fragmento)

```
sed -i 's/^bind-address.*/bind-address = 10.0.2.10/' /etc/mysql/mysql.conf.d/mysqld.cnf
systemctl restart mysql

# --- Simulación del NSG: solo la subred de la app puede llegar al 3306 ---
ufw --force reset
ufw default deny incoming
ufw default allow outgoing
ufw allow from 10.0.1.0/24 to any port 3306 proto tcp
ufw allow in on "$MGMT" to any port 22 proto tcp   # administración (vagrant ssh)
ufw --force enable
```

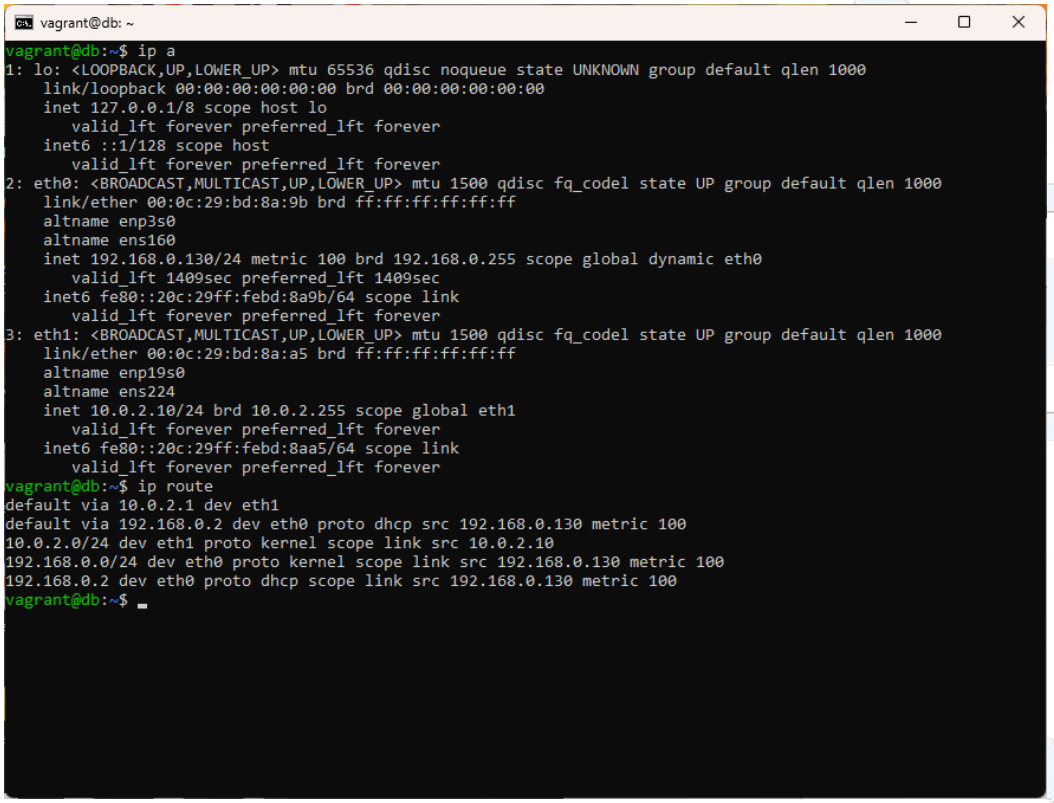


Figura 11. *La VM db con IP 10.0.2.10/24 y ruta por defecto vía 10.0.2.1.*

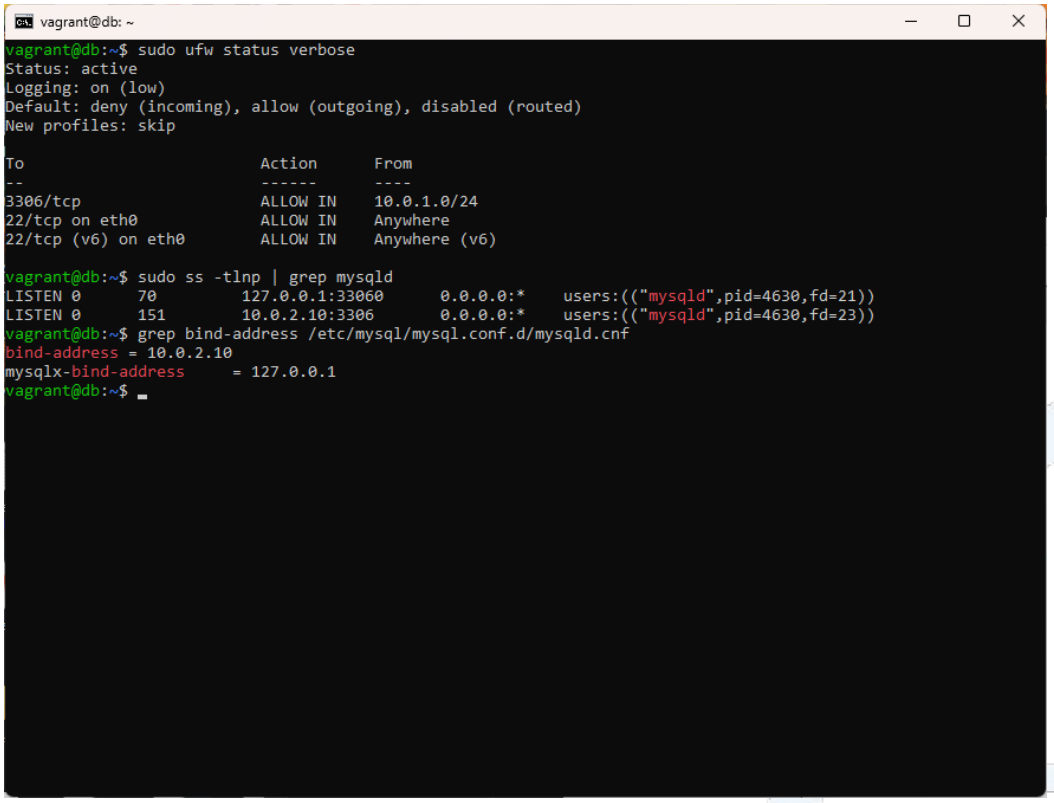


Figura 12. *UFW deniega la entrada por defecto y permite 3306/tcp solo desde 10.0.1.0/24; mysqld escucha únicamente en 10.0.2.10:3306.*

> **CORRECTO:** *El firewall permite el puerto del motor solo desde la subred de la aplicación, como pide la guía para el NSG.*

Base de datos y usuario con mínimo privilegio

Se creó la base de datos `empresa_db` con la tabla `clientes`. El usuario de la aplicación solo puede conectarse desde la subred `10.0.1.%` y solo tiene permisos sobre los datos.

provision/db.sh (fragmento SQL)

```
CREATE DATABASE IF NOT EXISTS empresa_db;
CREATE USER IF NOT EXISTS 'app_user'@'10.0.1.%' IDENTIFIED BY '********';
GRANT SELECT, INSERT, UPDATE, DELETE ON empresa_db.* TO 'app_user'@'10.0.1.%';

CREATE TABLE IF NOT EXISTS clientes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL,
  correo VARCHAR(120) NOT NULL UNIQUE
);
```

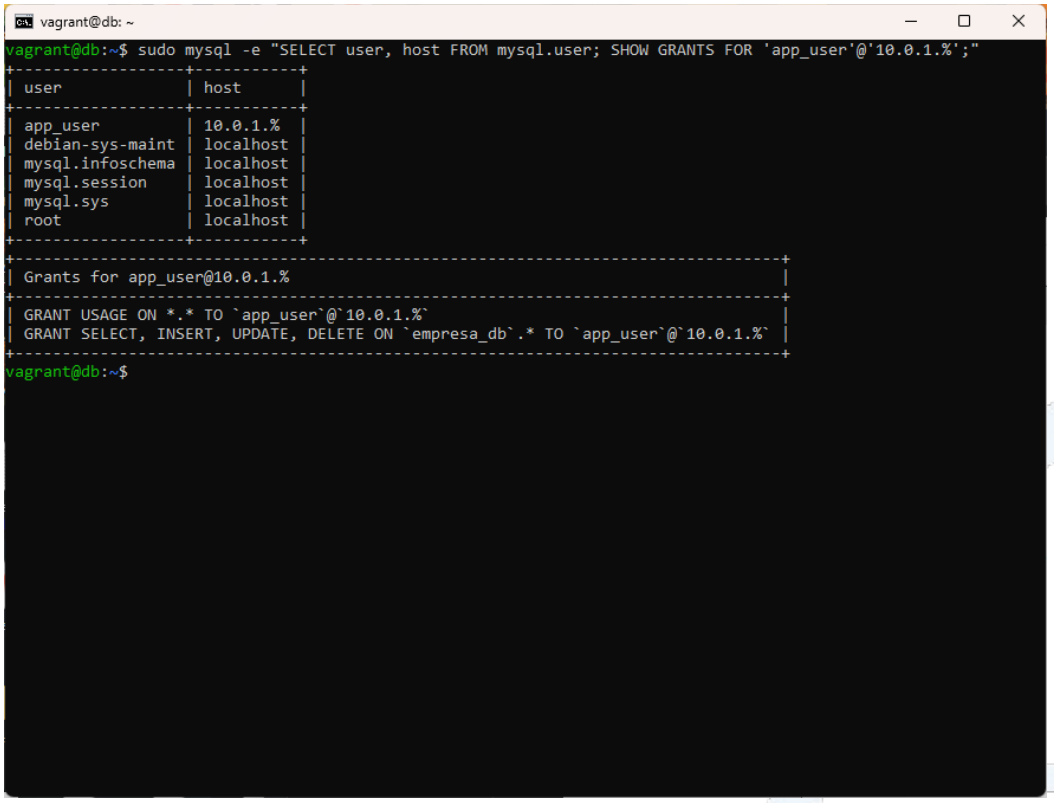


Figura 13. *El único usuario remoto es app_user@10.0.1.%, con permisos solo de datos; root existe solo en localhost.*

2.8. Conexión de la aplicación a la base de datos

Desde la VM app se conectó a MySQL usando la IP privada de la base de datos:

```
mysql -h 10.0.2.10 -u app_user -p empresa_db
```

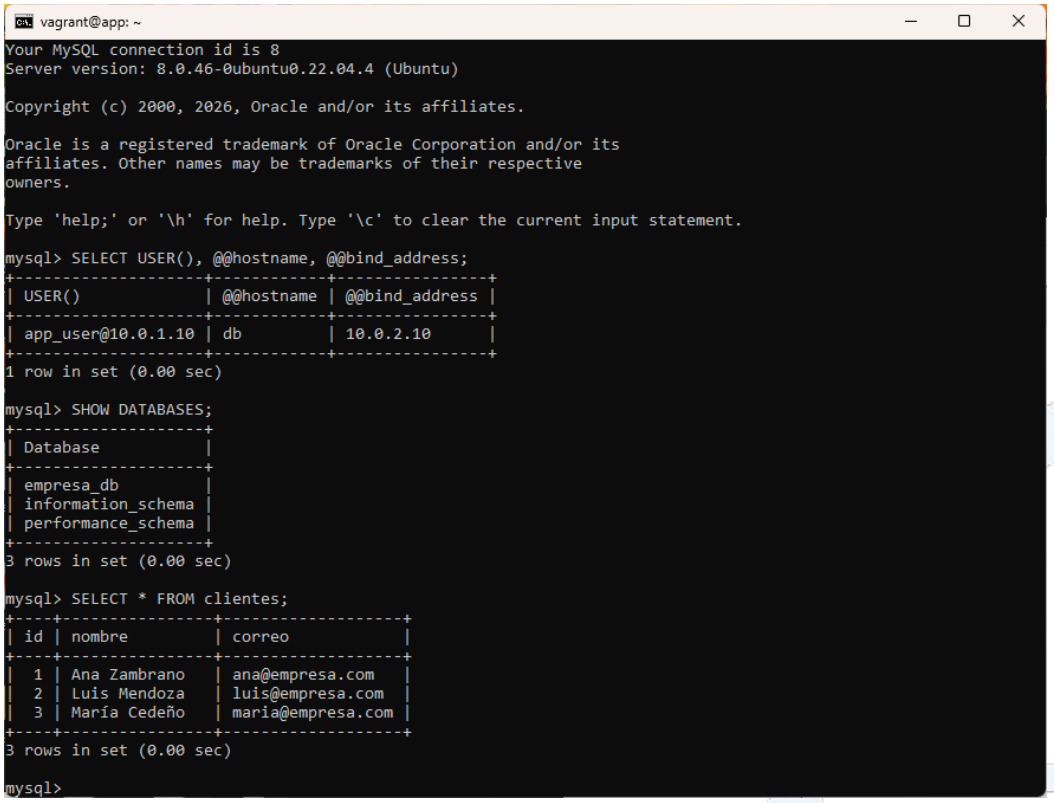


Figura 14. *Conexión desde app: USER() = app_user@10.0.1.10, el servidor es db con bind_address 10.0.2.10, y se consultan los registros de clientes.*

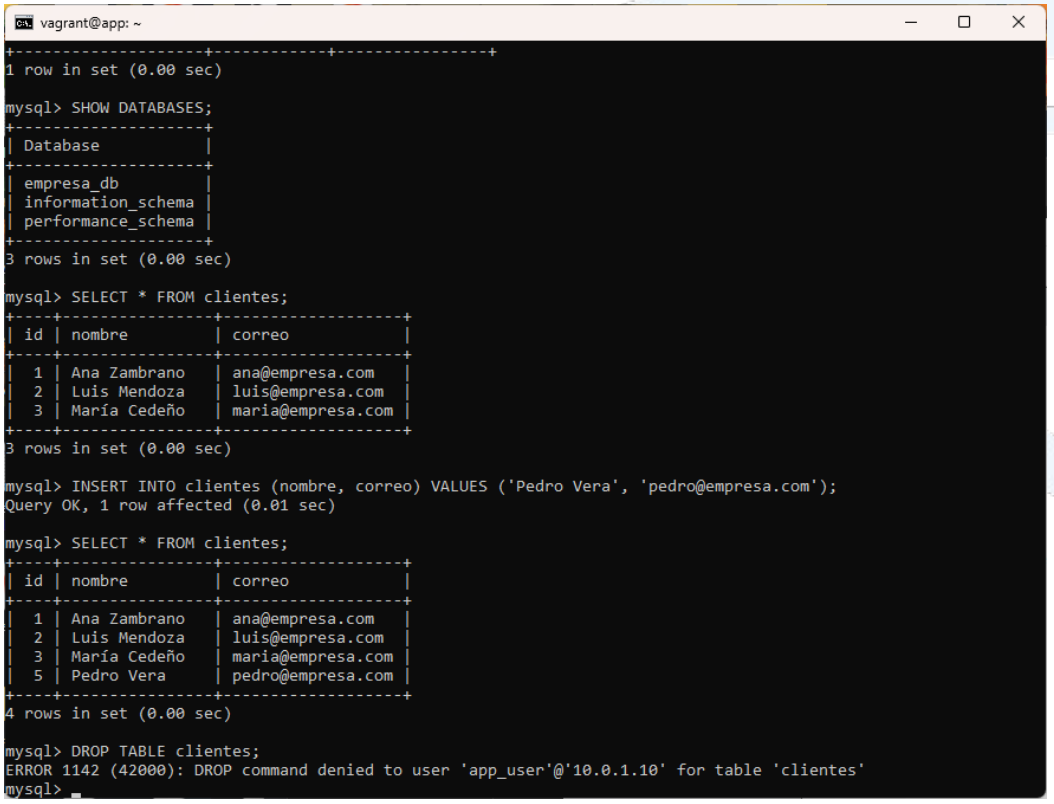


Figura 15. *El usuario de la aplicación puede insertar datos, pero DROP TABLE es rechazado (ERROR 1142).*

> **CORRECTO:** *La aplicación se conecta a la base de datos por la IP privada 10.0.2.10, con un usuario de mínimo privilegio.*

2.9. Pruebas de seguridad

Desde el router (está en la subred db, pero no en la subred app)

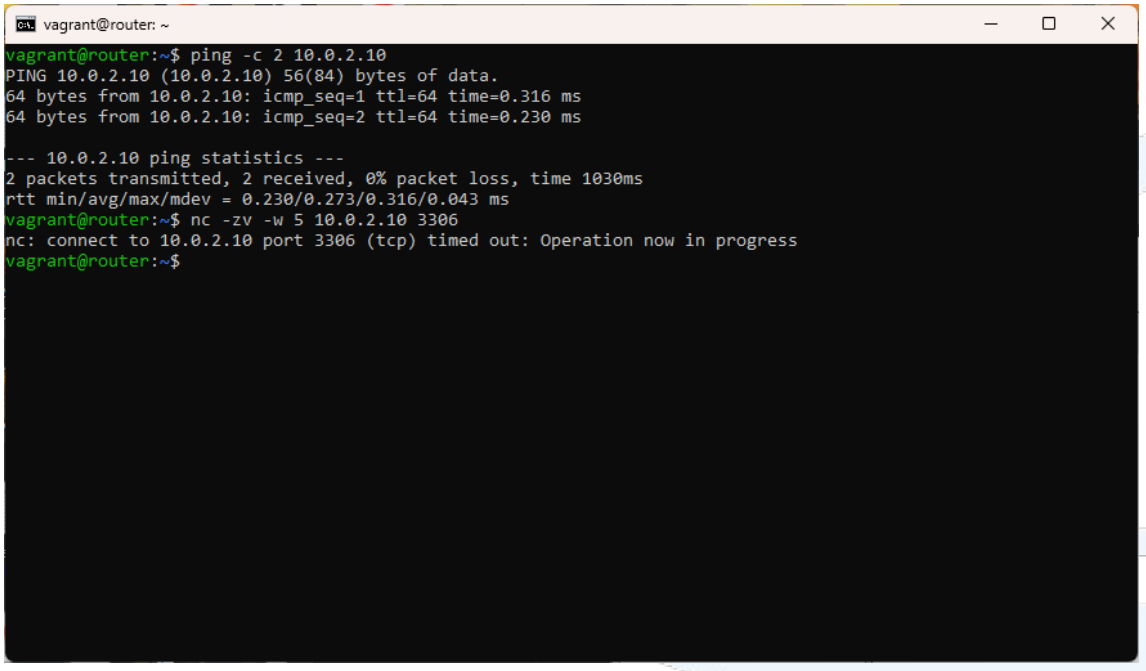


Figura 16. *El ping del router a db responde, pero la conexión al puerto 3306 expira: el firewall solo deja pasar a 10.0.1.0/24.*

Desde Windows (equivale a Internet en el laboratorio)

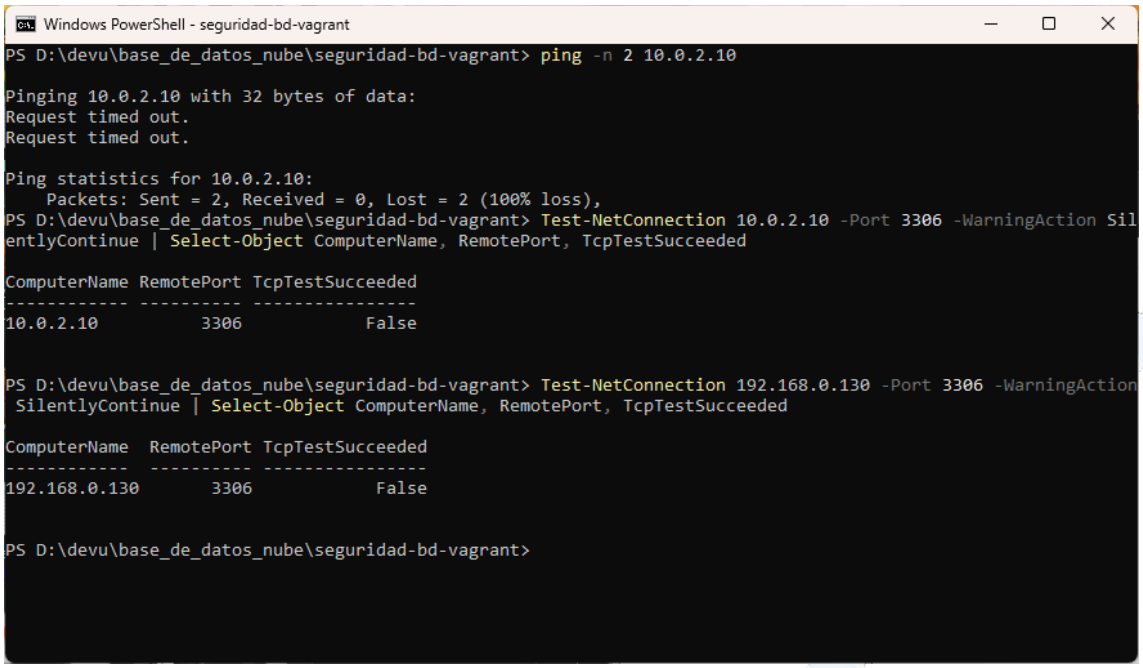


Figura 17. *Desde el equipo anfitrión no hay ping a 10.0.2.10, y TcpTestSucceeded = False tanto por la IP privada como por la IP de administración de db.*

Origen	Destino	Resultado
app 10.0.1.10	db 10.0.2.10 (ping)	Responde
app 10.0.1.10	db 10.0.2.10:3306	Conecta (regla UFW + usuario app_user)
app (app_user)	DROP TABLE clientes	Denegado (mínimo privilegio)
router 10.0.2.1	db 10.0.2.10:3306	Bloqueado (timeout)
Windows	db 10.0.2.10:3306	Sin ruta (subred privada)
Windows	db 192.168.0.130:3306	Bloqueado (MySQL no escucha ahí y UFW lo deniega)

> **CORRECTO:** *La base de datos solo es accesible desde la subred de la aplicación; cualquier otro origen queda bloqueado.*

Verificación automática

Para comprobar todo el laboratorio de una sola vez se preparó el script `pruebas/verificar.ps1`, que ejecuta 17 comprobaciones dentro de las VMs con `vagrant ssh` (rutas, ping, NAT, firewall, bind-address, usuario de MySQL, permisos y bloqueo del puerto 3306) y marca cada una como [OK] o [FALLO]. El paso a paso para repetir las pruebas manualmente está en `VERIFICACION.md`.

PowerShell

```
powershell -ExecutionPolicy Bypass -File pruebas\verificar.ps1
```

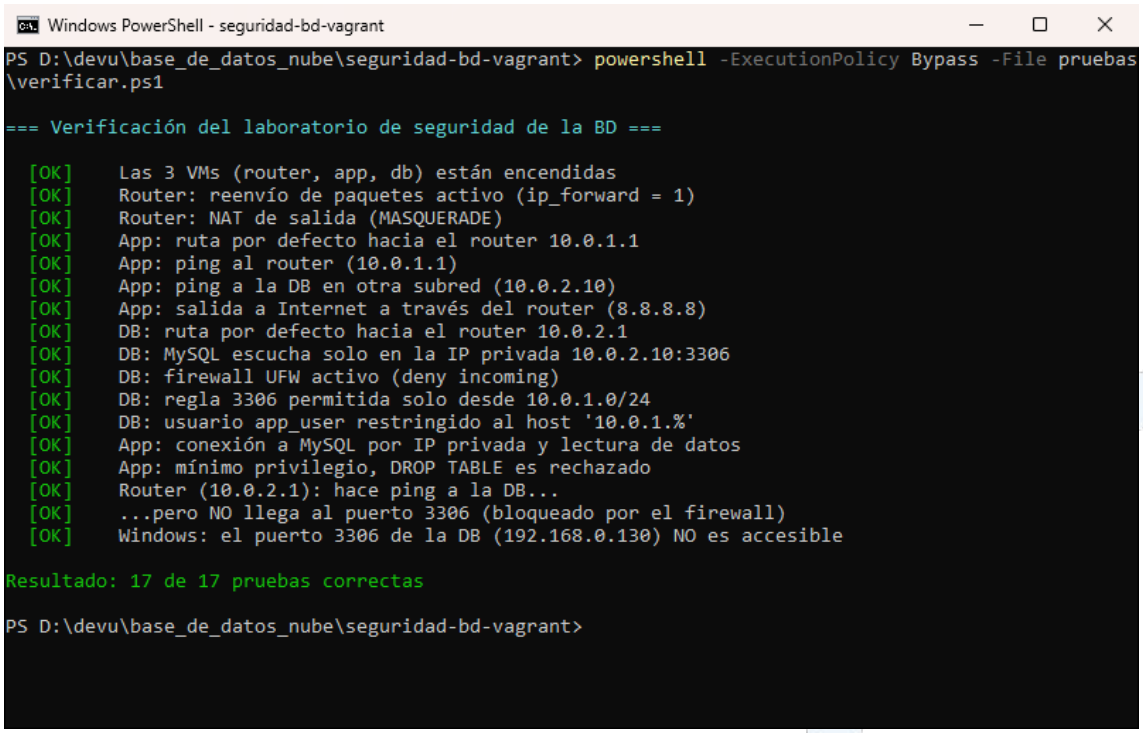


Figura 18. *Las 17 pruebas del laboratorio resultan correctas.*

> **CORRECTO:** *Resultado: 17 de 17 pruebas correctas.*

2.10. Buenas prácticas aplicadas

1. **Separar aplicación y base de datos en subredes distintas:** 10.0.1.0/24 y 10.0.2.0/24, en vmnets diferentes.
2. **Evitar exponer la base de datos a Internet:** MySQL escucha solo en 10.0.2.10, y esa subred no es alcanzable desde fuera.
3. **Limitar las reglas de seguridad por rango de IP:** UFW permite el 3306 solo desde 10.0.1.0/24, y MySQL acepta al usuario solo desde 10.0.1.%.
4. **Usar IP privadas para la comunicación interna:** la app se conecta con `mysql -h 10.0.2.10`.

2.11. Entregables

El proyecto se organizó en el repositorio `seguridad-bd-vagrant` para GitHub, con un README que explica cada paso:

Archivo	Contenido
`Vagrantfile`	Definición de las VMs router, app y db
`provision/*.sh`	Scripts de aprovisionamiento (router, NAT, rutas, app, db)
`host/crear_redes_vmware.ps1`	Creación de vmnet2 y vmnet3 en VMware
`pruebas/verificar.ps1`	Pruebas automáticas del laboratorio (17 comprobaciones)
`VERIFICACION.md`	Guía paso a paso para verificar y explicar el funcionamiento
`capturas/`	Evidencias de cada paso
`README.md`	Explicación completa de la práctica

3. Conclusiones

Hacer la práctica en local en vez de en Azure me sirvió para entender qué hace la nube por debajo: en Azure basta con crear la VNet y el NSG desde el portal, pero aquí tuve que montar yo mismo el router, las rutas por defecto y el firewall. Con eso me quedó claro que el NSG no es más que un filtro por IP de origen y puerto, y que la seguridad de verdad sale de combinar varias capas: la red separada, MySQL escuchando solo en la IP privada, el firewall y un usuario que solo puede entrar desde la subred de la aplicación y sin permiso para borrar tablas.

Los problemas que tuve fueron sobre todo del entorno. El Vagrantfile de la guía estaba hecho para VirtualBox, así que tuve que cambiar la box y la forma de crear las redes para VMware. El DHCP de VMware no arrancaba por una IP vieja guardada en el registro, y por eso las máquinas se quedaban esperando una dirección. Además, con 512 MB el router daba *kernel panic* al arrancar. Otra cosa que noté es que la guía bloquea todo el tráfico entrante de la base de datos sin dejar SSH, lo que deja a Vagrant sin acceso a la máquina.

La limitación que le veo al laboratorio es que Vagrant obliga a tener una interfaz NAT en cada máquina, así que la base de datos no queda totalmente "sin IP pública" como en Azure. Lo compensé haciendo que MySQL no escuche en esa interfaz y que el firewall solo deje pasar SSH por ella, y lo comprobé desde Windows. Aun así, me pareció una buena forma de practicar, porque todo queda en código y se puede volver a levantar con un solo `vagrant up`.